Abstract Factory

Abstract Factory · Light/Dark UI widget family stays consistent · Build with Waqar

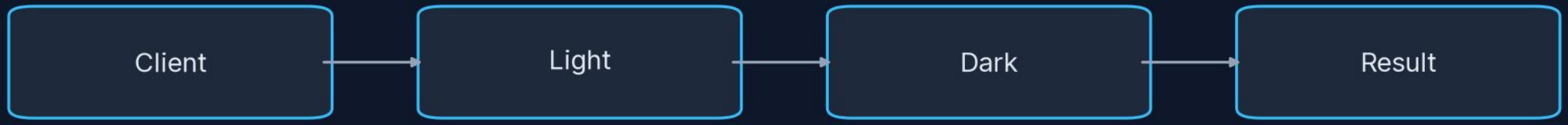
Today's Hook

- Today's force: Light/Dark UI widget family stays consistent
- We earn Abstract Factory by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Button, Checkbox, UiFactory, LightButton

The Problem

- Scenario: Light/Dark UI widget family stays consistent
- Constraints: solve «Light/Dark UI widget family stays consistent» with Abstract Factory — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Light/Dark UI widget family stays consistent
- Success looks like: light-button+dark-checkbox

Structure



Abstract Factory: Light/Dark UI widget family stays consistent

Real-World Scenario

- Running example: Light/Dark UI widget family stays consistent
- Key types: Button, Checkbox, UiFactory, LightButton
- Happy-path stdout cue: light-button+dark-checkbox
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

public final class NaiveUi {
  public static String render(String theme) {
    if ("light".equals(theme)) return "light-button+dark-checkbox"; // BUG: mixed family
    return "dark-button+light-checkbox";
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Light/Dark UI widget family stays consistent
- Without Abstract Factory, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

interface Button { String paint(); }
interface Checkbox { String paint(); }
interface UiFactory { Button button(); Checkbox checkbox(); }
record LightButton() implements Button { public String paint(){return "light-button";} }
record DarkButton() implements Button { public String paint(){return "dark-button";} }
record LightCheckbox() implements Checkbox { public String paint(){return "light-checkbox";} }
record DarkCheckbox() implements Checkbox { public String paint(){return "dark-checkbox";} }
record LightFactory() implements UiFactory {
  public Button button(){return new LightButton();} public Checkbox checkbox(){return new LightCheckbox();}
}
record DarkFactory() implements UiFactory {
  public Button button(){return new DarkButton();} public Checkbox checkbox(){return new DarkCheckbox();}
}
final class Dialog {
  private final UiFactory f; public Dialog(UiFactory f){this.f=f;}
  public String render(){ return f.button().paint()+"+"+f.checkbox().paint(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Abstract Factory: Light/Dark UI family ===");
    System.out.println("--- naive (mixed family bug) ---");
System.out.println(NaiveUi.render("light"));
System.out.println("--- abstract factory ---");
System.out.println(new Dialog(new LightFactory()).render());
System.out.println(new Dialog(new DarkFactory()).render());
  }
}
```

Walkthrough Output

```
=== Abstract Factory: Light/Dark UI family ===
--- naive (mixed family bug) ---
light-button+dark-checkbox
--- abstract factory ---
light-button+light-checkbox
dark-button+dark-checkbox
```

Step-by-Step Trace

- Match stdout to stages — look for: light-button+dark-checkbox
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 05-abstract-factory ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.gof.creational.abstractfactory.problem;

import java.util.*;

interface Button { String paint(); }
interface Checkbox { String paint(); }
interface UiFactory { Button button(); Checkbox checkbox(); }
record LightButton() implements Button { public String paint(){return "light-button";} }
record DarkButton() implements Button { public String paint(){return "dark-button";} }
record LightCheckbox() implements Checkbox { public String paint(){return "light-checkbox";} }
record DarkCheckbox() implements Checkbox { public String paint(){return "dark-checkbox";} }
record LightFactory() implements UiFactory {
  public Button button(){return new LightButton();} public Checkbox checkbox(){return new LightCheckbox();}
}
record DarkFactory() implements UiFactory {
  public Button button(){return new DarkButton();} public Checkbox checkbox(){return new DarkCheckbox();}
}
final class Dialog {
  private final UiFactory f; public Dialog(UiFactory f){this.f=f;}
  public String render(){ return f.button().paint()+"+"+f.checkbox().paint(); }
}
```

Trade-offs

- Use Abstract Factory when «Light/Dark UI widget family stays consistent» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Abstract Factory in product language.

Related Patterns

- Name the pattern usually confused with Abstract Factory and separate the forces.
- Sketch the roles for «Light/Dark UI widget family stays consistent» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Light/Dark UI widget family stays consistent
- Naive pain → Abstract Factory boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.